# 1 Проверка связи частотного и классического определения вероятности с помощью моделирования

**Вспомним**. ***Частотное определение вероятности*** построено на эксперименте и предполагает бесконечное количество испытаний.

Пусть эксперимент проводится $N$ раз ($N \to \infty$), $N_A$ – количества исходов, когда событие $A$ произошло, тогда вероятность события $A$ определяется как отношение $N_A$ к $N$ при $N \to \infty$:

$$P(A) = \lim_{N \to \infty} \frac{N_A}{N}$$

## 1.1 Задача (подбрасывание кубика)

Требуется оценить вероятность выпадения $1$ при подбрасывании игрального кубика с помощью классической и частотной вероятности (из $100000000$ испытаний).



Пользуясь классическим определением, получим:

$$p = \frac{1}{6} = 0.1(6)$$

Оценим частоту выпадения $1$ очка при $100000000$ подбрасываниях игрального кубика.

Ввод [53]:
```
import numpy as np

number_of_rolls = 1000000
dice_rolls = np.random.randint(1, 7, number_of_rolls)
len(dice_rolls[dice_rolls==1])/number_of_rolls   # len(np.where(dice_rolls == 1,
```

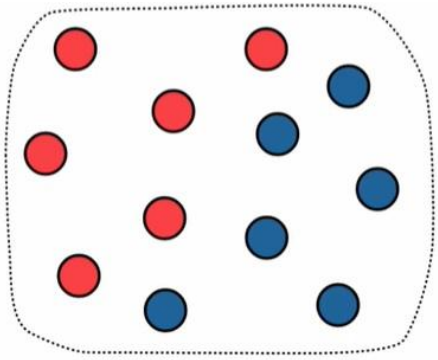
Out[53]:
```
0.166671
```

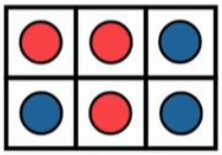
## 1.2 Задача (красные и синие шарики)

В коробке находится $6$ красных и $6$ синих шариков. Сколько существует способов вынуть $3$ красных и $3$ синих шарика?

Пользуясь классическим определением, получим:

$$p = \frac{C_6^3 \cdot C_6^3}{C_{12}^6} = 0.4329$$






Оценим вероятность вынуть $3$ красных и $3$ синих шарика, используя от $10$ до $1000$ испытаний.

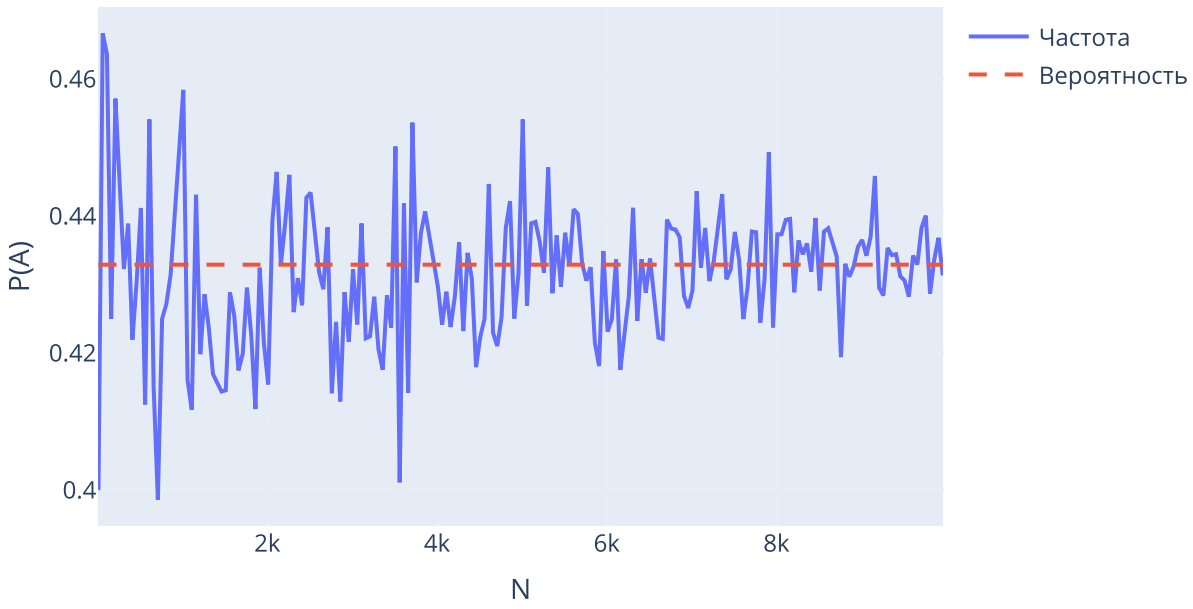
Ввод [7]:

```
from random import sample
import plotly.graph_objs as go
import numpy as np

def random_combination():
    urna = ['R'] * 6 + ['B'] * 6
    return sample(urna, 6)

N_values = np.array(range(10, 10001, 50))
P_A_values = []
for N in N_values:
    N_A = 0
    for _ in range(N):
        comb = random_combination()
        if comb.count('R') == 3:
            N_A += 1
    P_A_values.append(N_A / N)
print(round(P_A_values[-1], 4))
P = [0.4329] * len(N_values)
fig = go.Figure(layout=dict(width=600, height=300))
fig.update_layout(margin=dict(l=0, t=0, b=0))
fig.add_trace(go.Scatter(x=N_values, y=P_A_values, name='Частота'))
fig.add_trace(go.Scatter(x=N_values, y=P,
                         name='Вероятность', line_dash='dash'))
fig.update_xaxes(title_text='N')
fig.update_yaxes(title_text='P(A)')
fig.show();
```

```
0.4313
```



## 1.3 Задача (сумма на двух кубиках)

Требуется оценить вероятности всех возможных значений суммы очков, выпадающей на двух игральных костях при одновременном подбрасывании, с помощью классической и частотной вероятности (из $1000$ испытаний).

Ввод [67]:

```
# классическая вероятность
from itertools import product

ls = list(product('123456', repeat=2))

prob = {}   # словарь сумма - вероятность
rec = {s: [] for s in range(2, 13)}
for pair in ls:
    sm = sum(list(map(int, pair)))
    prob[sm] = prob.get(sm, 0) + 1
    rec[sm].append('+'.join(pair))

N = len(ls)
for sm, cnt in prob.items():
    prob[sm] = cnt / N
    variants = '='.join(rec[sm])
    print(f'Сумма {sm}: {variants}, вероятность: {prob[sm]:.4f}.')
```

```
Сумма 2: 1+1, вероятность: 0.0278.
Сумма 3: 1+2=2+1, вероятность: 0.0556.
Сумма 4: 1+3=2+2=3+1, вероятность: 0.0833.
Сумма 5: 1+4=2+3=3+2=4+1, вероятность: 0.1111.
Сумма 6: 1+5=2+4=3+3=4+2=5+1, вероятность: 0.1389.
Сумма 7: 1+6=2+5=3+4=4+3=5+2=6+1, вероятность: 0.1667.
Сумма 8: 2+6=3+5=4+4=5+3=6+2, вероятность: 0.1389.
Сумма 9: 3+6=4+5=5+4=6+3, вероятность: 0.1111.
Сумма 10: 4+6=5+5=6+4, вероятность: 0.0833.
Сумма 11: 5+6=6+5, вероятность: 0.0556.
Сумма 12: 6+6, вероятность: 0.0278.
```

Ввод [4]:

```python
# частотная вероятность
from random import choices

def random_combination():
    return choices([1, 2, 3, 4, 5, 6], k=2)

# print(random_combination())

N = 100_000
freq = {}    # словарь сумма - частота
for i in range(N):
    comb = random_combination()
    sm = sum(comb)
    freq[sm] = freq.get(sm, 0) + 1

for sm in sorted(freq.keys()):
    freq[sm] /= N
    print(f'Сумма {sm}, частота: {freq[sm]:.4f}')
```

```
Сумма 2, частота: 0.0286
Сумма 3, частота: 0.0562
Сумма 4, частота: 0.0822
Сумма 5, частота: 0.1095
Сумма 6, частота: 0.1393
Сумма 7, частота: 0.1677
Сумма 8, частота: 0.1390
Сумма 9, частота: 0.1100
Сумма 10, частота: 0.0822
Сумма 11, частота: 0.0573
Сумма 12, частота: 0.0279
```

Ввод [5]:

```
import pandas as pd

frequency = [freq[sm] for sm in sorted(freq.keys())]
df = pd.DataFrame(index=prob.keys(),
                  data={'Вероятность': prob.values(),
                        'Частота': frequency
                       }
                 )

df['Отклонение'] = abs(df['Вероятность'] - df['Частота'])
df
```

Out[5]:

| | Вероятность | Частота | Отклонение |
|---|---|---|---|
| **2** | 0.027778 | 0.02865 | 0.000872 |
| **3** | 0.055556 | 0.05616 | 0.000604 |
| **4** | 0.083333 | 0.08220 | 0.001133 |
| **5** | 0.111111 | 0.10949 | 0.001621 |
| **6** | 0.138889 | 0.13934 | 0.000451 |
| **7** | 0.166667 | 0.16771 | 0.001043 |
| **8** | 0.138889 | 0.13898 | 0.000091 |
| **9** | 0.111111 | 0.11005 | 0.001061 |
| **10** | 0.083333 | 0.08219 | 0.001143 |
| **11** | 0.055556 | 0.05734 | 0.001784 |
| **12** | 0.027778 | 0.02789 | 0.000112 |

Ввод [7]:

```
df[['Вероятность', 'Частота']].plot.bar(rot=0)
```

Out[7]: <Axes: >

